PRODUCT REQUIREMENTS DOCUMENT

GlobalRisk AI

LLM-Powered Financial Disclosure Intelligence Platform

Hackathon Edition · 15-Hour Build Plan · 4-Member Team

Group 12	AIML Dept.	SIES GST
Riya · Chhaya · Stuti · Sakshi	**Major Project**	**2025–26**

■ Build Time	■ Team Size	■ Phases	■ Demo Target
15 Hours	**4 Members**	**4 Phases**	**3 Minutes**

1. PROJECT OVERVIEW

GlobalRisk AI is an AI-powered financial disclosure intelligence platform that automatically analyzes and compares annual corporate reports from multiple financial markets (SEC–USA, NSE/BSE–India). It uses LLMs, RAG, semantic embeddings, and NLP to surface newly introduced risks, removed disclosures, tone shifts, and severity changes — transforming hundreds of pages of legalese into actionable insights in seconds.

■ **Problem**	Annual reports (300–500 pages) are impossible to compare manually. No unified AI tool exists for US+India markets.
■ **Solution**	Automated LLM pipeline for year-over-year semantic diff of 10-K, 10-Q, and DRHP filings with explainable risk scoring.
■ **Users**	Financial analysts, institutional investors, regulators, research teams, compliance officers.
■ **WOW Factor**	Live demo: upload two PDFs → 30-second AI-powered risk delta with a visual heatmap. No competitor does this cross-market.

2. CORE FEATURES (MVP)

F1	**PDF Ingestion & Parsing** Upload any SEC 10-K / DRHP PDF. Extract text by section (Risk Factors, MD&A;, Financials) using pdfplumber. Support 2-PDF comparison (Year A vs Year B).
F2	**Semantic Chunking & Embedding** Chunk documents into 512-token overlapping windows. Generate sentence embeddings via sentence-transformers (all-MiniLM-L6-v2). Store in FAISS vector index for sub-second retrieval.
F3	**Risk Delta Detection** Compare embedding similarity scores year-over-year. Flag: NEW risks (cosine sim < 0.4), REMOVED risks, AMPLIFIED risks (severity keywords). Output structured JSON diff.
F4	**LLM-Powered Explanation** Pass top-k changed chunks to GPT-3.5-turbo / Llama-3 via RAG pipeline. Generate plain-English explanations: 'Why did this risk appear?' 'What changed in tone?'
F5	**Risk Dashboard UI** React/Next.js dashboard: Risk heatmap, timeline slider (Year A→B), severity score (0–10), side-by-side text diff with highlighted changes.
F6	**Cross-Market Support** Normalize SEC (USA) and BSE/NSE (India) report structures. Support multi-company comparison. Company search via ticker/name.

3. TECHNICAL ARCHITECTURE

Layer	Technology	Responsibility
Frontend	Next.js 14 + Tailwind CSS + shadcn/ui	React hooks, Recharts for risk visualization, PDF.js for viewer
Backend API	FastAPI (Python 3.11)	REST endpoints: /upload, /compare, /query, /companies
AI Pipeline	LangChain + sentence-transformers + OpenAI / Ollama	RAG pipeline: chunk → embed → retrieve → generate
Vector Store	FAISS (in-memory, persist to disk)	Index per document, cosine similarity search, top-k retrieval
PDF Processing	pdfplumber + PyMuPDF	Section-aware extraction, table parsing, metadata capture
Database	Supabase (PostgreSQL)	Document registry, risk snapshots, company master, user sessions
Storage	Supabase Storage	Raw PDF store, processed chunk cache
Deployment	Vercel (frontend) + Railway / HuggingFace Spaces (API)	Free-tier stack, no GPU needed for demo

4. PHASE-WISE DEVELOPMENT PLAN (15-HOUR HACKATHON)

PHASE 1 · Foundation & Data Pipeline

■ 0h – 3h 30m

Goal: Working PDF parser + FAISS vector index + Supabase schema live.

- ■ Setup monorepo: /frontend (Next.js), /backend (FastAPI), /ai (pipeline scripts)
- ■ Install & configure: pdfplumber, sentence-transformers, FAISS, langchain, supabase-py
- ■ Build PDF ingestion API: POST /upload → extract text → section-split → chunk (512 tokens, 50 overlap)
- ■ Generate embeddings: all-MiniLM-L6-v2 — fast, free, no GPU needed
- ■ Store embeddings in FAISS; persist index + raw chunks to Supabase Storage
- ■ Supabase schema: documents, chunks, companies, risk_snapshots tables
- ■ Test with 2 sample 10-K PDFs (Apple 2022 vs 2023 from SEC EDGAR)

■ DELIVERABLE — CLI: python ingest.py apple_2022.pdf apple_2023.pdf → prints chunk count & index saved ✓

PHASE 2 · Risk Delta Engine (AI Core)

■ 3h 30m – 7h

Goal: Accurate year-over-year risk diff with LLM explanations running end-to-end.

- ■ Build compare engine: load FAISS indexes for Year A & B → pairwise cosine similarity
- ■ Risk classification logic: NEW (sim < 0.4), REMOVED (only in A), AMPLIFIED (sim 0.4–0.7 + severity keywords like 'material', 'significant', 'adverse')

- Severity keyword list: {'critical','material','significant','adverse','unprecedented','severe','regulatory','lawsuit','breach'}
- Build RAG chain: LangChain RetrievalQA → retrieve top-5 chunks → GPT-3.5-turbo prompt
- Prompt template: 'You are a financial risk analyst. Given these disclosure excerpts from {company} annual reports ({year_a} vs {year_b}), explain the key risk changes in 3 bullet points.'
- Implement /compare API: POST {doc_id_a, doc_id_b} → returns {new_risks, removed, amplified, llm_summary}
- Add Ollama fallback (llama3:8b) for offline/rate-limit scenarios
- Unit test: compare outputs match expected risk categories on test PDFs

DELIVERABLE API returns structured JSON with risks + LLM summary. Postman test passes ✓

PHASE 3 · Frontend Dashboard

7h – 11h 30m

Goal: Polished, demo-ready UI that wows judges in 3 minutes.

- Build 3-screen flow: Landing → Upload (drag-drop PDFs) → Results Dashboard
- Results Dashboard layout: top risk score card (0–10) + risk delta summary
- Risk Heatmap: Recharts heatmap — severity on Y-axis, risk category on X-axis, intensity as color
- Side-by-side text diff: highlighted NEW (green), REMOVED (red), CHANGED (amber) paragraphs
- LLM summary card with streaming text (SSE from FastAPI)
- Company search autocomplete (NSE/BSE + SEC tickers hardcoded for demo)
- Year selector: slider A | B → triggers compare API
- Loading states with progress indicators (chunk extraction → embedding → comparison → LLM)
- Responsive design; dark mode toggle; Tailwind + shadcn/ui components

DELIVERABLE Full UI flow working: upload → 30s processing → dashboard with heatmap ✓

PHASE 4 · Polish, Demo Prep & Submission

11h 30m – 15h

Goal: Zero bugs during demo. Judges remember your project.

- Pre-load 4 demo companies: Apple (SEC), Infosys (BSE), Tesla (SEC), Reliance (BSE) — indexes ready
- Happy path rehearsal: run demo flow 5× — measure time, fix bottlenecks
- Add export: Download PDF Report button (generates summary PDF via reportlab)
- Error handling: timeout fallbacks, empty state messages, loading skeletons
- Performance: cache FAISS queries, pre-generate LLM summaries for demo companies
- Deployment: Vercel frontend live URL; backend on Railway or HuggingFace Spaces
- Prepare 3-min demo script: problem → live upload → AI comparison → insights → market impact
- Prepare slide deck: 6 slides max — problem, solution, tech stack, live demo, traction, team
- Record 90-second screen-record backup in case of live demo failure

DELIVERABLE Live URL + working demo + submission package ready ✓

5. TEAM MEMBER TASK ASSIGNMENTS

Riya Chaurasiya (123A9006) — AI/ML Lead — Phase 1 + Phase 2

Focus: RAG pipeline, embedding engine, risk delta logic

- Setup LangChain + sentence-transformers environment
- Build PDF text extraction & section-splitting logic (pdfplumber)
- Implement chunking strategy: 512-token windows with 50-token overlap
- Generate FAISS index per document; persist to disk
- Build compare engine: pairwise cosine similarity + risk classification
- Tune severity keyword dictionary for financial domain
- Write LangChain RAG chain with GPT-3.5 / Llama3 fallback
- Optimize for speed: target < 30s for demo comparison
- Unit tests for risk classification accuracy

■ TOOLS Python, LangChain, FAISS, sentence-transformers, OpenAI API, pdfplumber

Chhaya Gami (123A9007) — Backend Engineer — Phase 1 + Phase 2 + Phase 4

Focus: FastAPI server, Supabase integration, deployment

- Setup FastAPI project structure with routers, models, dependencies
- Design Supabase schema: documents, chunks, companies, risk_snapshots
- Implement /upload endpoint: receive PDF → trigger AI pipeline → store results
- Implement /compare endpoint: JSON response with full risk delta
- Implement /query endpoint: free-text Q&A; against any stored report
- Add streaming SSE endpoint for LLM summary (real-time feel in UI)
- CORS setup, error handling, rate limiting for demo stability
- Deploy backend to Railway / HuggingFace Spaces with env vars
- Write API docs (auto-generated Swagger via FastAPI)

■ TOOLS Python, FastAPI, Supabase, PostgreSQL, Railway/HuggingFace, uvicorn

Stuti Chilweri (123A9008) — Frontend Engineer — Phase 3 + Phase 4

Focus: Next.js dashboard, data visualization, UX polish

- Setup Next.js 14 (App Router) + Tailwind CSS + shadcn/ui
- Build Landing page: hero, value props, CTA to upload
- Build Upload page: drag-drop for 2 PDFs + company/year selector + progress bar
- Build Results Dashboard: risk score card, heatmap (Recharts), diff view
- Implement SSE streaming for LLM summary with typewriter animation
- Side-by-side text diff component: color-coded NEW/REMOVED/CHANGED
- Company search autocomplete with preset ticker list
- Dark mode toggle, responsive layout (desktop-first)
- Deploy to Vercel; configure environment variables for API URL

■ TOOLS Next.js 14, React, Tailwind CSS, shadcn/ui, Recharts, Vercel

Sakshi Kale (123A9016)

Data & Demo Strategist

Phase 1 + Phase 4

Focus: Data pipeline, demo preparation, presentation, export features

- Curate 4 demo company dataset: download 10-K PDFs from SEC EDGAR & BSE portal
- Pre-process and ingest all 8 PDFs (2 years × 4 companies) → indexes ready for demo
- Validate risk outputs: manually verify 3 known risk changes are caught by AI
- Build PDF export feature: reportlab generates 1-page risk summary PDF
- Prepare 3-minute demo script with talking points for each UI screen
- Create 6-slide pitch deck: problem, solution, tech, live demo, impact, team
- Record 90-second screen-record backup for fallback
- Prepare judges Q&A; answers: accuracy, scalability, monetization, future roadmap
- Write README.md and submission form content

TOOLS Python, reportlab, SEC EDGAR API, BSE portal, PowerPoint/Canva, OBS Studio

6. 15-HOUR TIMELINE AT A GLANCE

Hour	Phase	Riya (AI/ML)	Chhaya (Backend)	Stuti (Frontend)	Sakshi (Data)
0–3.5h	PHASE 1	Chunking + FAISS index	FastAPI setup + Supabase schema	Next.js scaffold + landing page	PDF curation + ingest test
3.5–7h	PHASE 2	RAG chain + risk delta engine	/compare + /query endpoints	Upload page + progress UI	Risk output validation
7–11.5h	PHASE 3	LLM tuning + accuracy	SSE streaming endpoint	Dashboard + heatmap + diff view	Pre-load demo companies
11.5–15h	PHASE 4	Speed optimization	Deploy + error handling	Deploy Vercel + polish	Demo script + pitch deck

7. HACKATHON OPTIMIZATION STRATEGY

■ Judge WOW Moments

- ■ Open with the problem: 'Analysts spend 3 weeks comparing reports. We do it in 30 seconds.'
- ■ Live upload a real Apple 10-K PDF on screen — no mock data
- ■ Show the risk heatmap appearing in real time as LLM streams
- ■ Demonstrate a genuine risk they'd never spot manually (e.g., new cybersecurity language in 2023)
- ■ End with: 'This works for any public company in US and India markets, right now.'

■ Speed & Reliability

- ■ Pre-index all 4 demo companies before presentation — comparison should take < 15 seconds
- ■ Use smaller LLM (gpt-3.5-turbo, not gpt-4) for speed; Ollama offline fallback
- ■ Cache LLM summaries for demo companies to avoid API latency
- ■ Test demo flow 10× the night before; have backup screen recording
- ■ Deploy on stable infra; avoid localhost during presentation

■ Visual Excellence

- ■ Risk heatmap: judges remember visuals, not text — make it colorful and clear
- ■ Smooth loading animations with meaningful progress steps
- ■ Color-coded diff view: green = new risk, red = removed, amber = changed
- ■ Dark mode enabled by default — looks more professional on stage projectors
- ■ Company logos in search autocomplete — small touch, big impression

■ Technical Depth (for Q&A;)

- ■ Know your F1 score: test on 20 known risk changes, report accuracy
- ■ Explain RAG vs fine-tuning trade-off (why RAG wins for this use case)
- ■ Scaling answer: 'FAISS can handle 10M+ chunks; Supabase scales to enterprise'
- ■ Monetization: SaaS at $299/month per analyst, or API licensing to Bloomberg/Refinitiv
- ■ Research gap you solve: first unified US+India cross-market risk intelligence tool

8. FULL TECH STACK

Component	Technology	Why
LLMs	GPT-3.5-turbo (primary), Llama-3 8B via Ollama (fallback)	Free tier + local
Embeddings	sentence-transformers/all-MiniLM-L6-v2	Free, CPU-fast, 384-dim
Vector DB	FAISS (in-memory + disk persist)	Zero cost, sub-ms search
RAG Framework	LangChain 0.2	RetrievalQA + streaming
PDF Processing	pdfplumber + PyMuPDF	Section-aware extraction
Backend	FastAPI + uvicorn	Async, auto-docs, SSE support
Database	Supabase (PostgreSQL)	Auth + storage + RLS
Frontend	Next.js 14 (App Router)	SSR, fast, easy deploy
UI Components	shadcn/ui + Tailwind CSS	Professional look fast
Charts	Recharts	React-native, customizable
Export	reportlab	PDF summary generation
Deployment	Vercel + Railway	Free tier, < 5 min deploy

9. RESEARCH GAPS ADDRESSED

Gap 1: Single-market tools	Existing systems (FinBERT, FinGPT) are trained/tested only on US SEC filings. GlobalRisk AI natively supports NSE/BSE India filings with normalized section mapping.
Gap 2: No year-over-year semantic diff	Literature review models do sentiment analysis on a single document. GlobalRisk AI computes pairwise embedding similarity to detect temporal disclosure drift.
Gap 3: No risk emergence detection	Current tools miss NEW risks (never mentioned before). Our cosine-threshold classifier specifically surfaces zero-shot emerging risks.
Gap 4: Black-box outputs	FinGPT outputs predictions without explanation. GlobalRisk AI provides chunk-level citations and LLM-generated rationale for every flagged change.
Gap 5: No unified cross-standard platform	SEC (GAAP) and BSE (IndAS) use different section structures. Our normalization layer maps both to a unified risk taxonomy.